

FEATURE-STANDARD — Standard di Documentazione Feature

Standard riutilizzabile su tutti i progetti NGS. Seguire questo documento all'apertura di ogni branch feature/hotfix.

Perché documentare durante lo sviluppo?

I tool di generazione automatica ([docs/docs-generator/generate-history-docs.py](#), [docs/docs-generator/run-with-ai.py](#)) producono CHANGELOG e README di qualità proporzionale alla qualità delle informazioni disponibili:

Informazione disponibile	Qualità output AI
Solo file modificati	Scarsa — descrizioni generiche
Commit message descrittivi	Buona — sezioni Added/Fixed/Changed corrette
Conventional Commits	Ottima — raggruppamento automatico per tipo
FEATURE.md + Conventional Commits	Eccellente — contesto intento + motivazioni

Tier 1 — Minimo (commit message descrittivi)

Scrivere sempre commit message che rispondano a "cosa fa questo commit":

```
# BAD
fix bug
update
wip
minor changes

# GOOD
add alarm threshold configuration per ECB type
fix sunrise offset calculation for Crotone airport
update dotnet-script version in effemeridi-converter
remove duplicate lamp group entries from ECB_A config
```

Regola: almeno 8-10 parole, soggetto-verbo-oggetto, in inglese o italiano ma coerente.

Tier 2 — Raccomandato (Conventional Commits)

Cos'è Conventional Commits?

[Conventional Commits](#) è uno standard per i commit message basato su una struttura semplice:

```
<type>[optional scope]: <description>

[optional body]

[optional footer(s)]
```

Tipi (type)

Tipo	Quando usarlo	Sezione CHANGELOG
feat	Nuova funzionalità	Added
fix	Correzione di un bug	Fixed
docs	Solo documentazione	(opzionale)
chore	Manutenzione, dipendenze, config	(opzionale)
refactor	Refactoring senza cambi comportamento	Changed
perf	Miglioramento prestazioni	Changed
test	Aggiunta/modifica test	(opzionale)
style	Formattazione, spazi, nomi	(opzionale)
ci	Pipeline CI/CD	(opzionale)
revert	Revert di un commit precedente	Removed
security	Fix vulnerabilità	Security

Scope (opzionale ma utile)

Lo scope indica il modulo/componente coinvolto:

```
feat(ccr-alarm-converter): add batch decode mode
fix(effemeridi-converter): correct february boundary calculation
chore(SoftwareVersionManager): update ClosedXML to 0.96.0
docs(git-history-docs): add FEATURE-STANDARD reference to GUIDE.md
```

Esempi concreti

```
# Feature nuova
git commit -m "feat(ilcms): add TB-2 configuration support for Fiumicino airport"

# Bug fix
git commit -m "fix(ccr-alarm-converter): handle null value in bitfield when CCR
offline"

# Breaking change (aggiungere ! dopo il tipo/scope)
git commit -m "feat(SoftwareVersionManager)!: change composite version format to
semver

BREAKING CHANGE: SixDigitBuildVersion format changed from XXXXXX to X.X.X.X"

# Chore senza scope
git commit -m "chore: add __pycache__ to .gitignore"

# Documentazione
git commit -m "docs(ngs-test-analyzer): add vocal protocol PDF export
instructions"
```

Perché usarlo?

- Il generatore `generate-history-docs.py` può raggruppare automaticamente `feat:` → Added, `fix:` → Fixed
 - Genera automaticamente la sezione `[v0.9.0]` del CHANGELOG per versione
 - Strumenti come `semantic-release` possono fare bump automatico di versione
 - La history è leggibile da qualsiasi sviluppatore senza contesto pregresso
-

Tier 3 — Ideale (FEATURE.md sul branch)

Creare FEATURE.md all'apertura del branch

Alla creazione di ogni `feature/*` o `hotfix/*`, creare un file `FEATURE.md` nella root del repository:

```
git checkout -b feature/nghiselli/nome-feature develop
# Creare subito FEATURE.md
```

Template FEATURE.md

```
# Feature: <nome-feature>

**Branch:** `feature/<owner>/<nome-feature>`
**Aperta:** YYYY-MM-DD
**Autore:** <nome>

## Obiettivo
<!-- Una frase: cosa fa questa feature e perché esiste. -->

## Scope

### Incluso
-

### Escluso (out of scope)
-

## Decisioni tecniche
<!-- Annotare scelte non ovvie e la motivazione. -->
| Decisione | Alternativa scartata | Motivazione |
|-----|-----|-----|
|          |          |          |

## Note di sviluppo
<!-- Aggiornare man mano durante lo sviluppo. -->

- YYYY-MM-DD:

## Breaking changes
<!-- Elencare eventuali breaking changes che richiedono aggiornamenti da parte dei
consumer. -->
Nessuno / oppure:
-

## Link utili
<!-- Issue, PR, documenti tecnici, specifiche. -->
-
```

Comportamento al merge

FEATURE.md non va in main. Due opzioni:

Opzione A (raccomandato): Spostarlo in `docs/features/<nome-feature>.md` prima del merge

```
git mv FEATURE.md docs/features/nome-feature.md
git commit -m "docs: archive feature notes for nome-feature"
```

Opzione B: Eliminarlo prima del merge — il contenuto è già stato usato dall'AI per generare la documentazione.

Perché funziona con il tool?

`run-with-ai.py` chiama `collect_repo_context()` che — con questa estensione — legge automaticamente:

- `README.md` (root)
- `CHANGELOG.md` (root)
- `FEATURE.md` (se presente sul branch corrente)
- `README.md` di ogni sotto-modulo

Il modello AI riceve quindi sia la *storia* (commit) sia l'*intento* (FEATURE.md), producendo descrizioni molto più accurate e contestualizzate.

Tier 4 — Automatizzato (commit-msg hook)

Installare il hook (opzionale)

Crea il file `.git/hooks/commit-msg` (non versionato, da installare su ogni clone):

```
#!/bin/sh
# Valida il formato Conventional Commits
MSG=$(head -1 "$1")
PATTERN="^(feat|fix|docs|chore|refactor|perf|test|style|ci|revert|security)(\
(.+\\))?!?: .{10,}"

if ! echo "$MSG" | grep -qE "$PATTERN"; then
  echo ""
  echo "ERROR: Commit message non segue il formato Conventional Commits."
  echo ""
  echo "  Formato: <type>[(<scope>)]: <description>"
  echo "  Esempio: feat(ccr-alarm-converter): add batch decode mode"
  echo ""
  echo "  Tipi validi: feat fix docs chore refactor perf test style ci revert
security"
  echo ""
  exit 1
fi
```

Windows: Git for Windows include Git Bash, che esegue automaticamente gli hook `#!/bin/sh`. Non serve `chmod` né configurazioni aggiuntive — basta copiare il file in `.git/hooks/commit-msg`.

```
# Linux/macOS – rendere eseguibile dopo la copia
chmod +x .git/hooks/commit-msg
```

Installazione condivisa via script (raccomandato)

Aggiungere al repository i file dal boilerplate:

```
scripts/
  hooks/
    commit-msg          ← copia da boilerplate/project-
standard/tools/hooks/commit-msg
  install-hooks.ps1     ← copia da boilerplate/project-standard/tools/install-
hooks.ps1
```

Eseguire dalla root del progetto (Windows, Linux, macOS):

```
# PowerShell (Windows / cross-platform)
.\scripts\install-hooks.ps1
```

Lo script individua automaticamente la root git, copia tutti i file da `scripts/hooks/` in `.git/hooks/`, e supporta hook aggiuntivi futuri senza modifiche.

Checklist apertura feature

```
[ ] git checkout -b feature/<owner>/<nome> develop
[ ] Creare FEATURE.md dalla sezione Template qui sopra
[ ] git add FEATURE.md && git commit -m "docs: add FEATURE.md for <nome-feature>"
[ ] (opzionale) Installare commit-msg hook: .\scripts\install-hooks.ps1
```

Checklist chiusura feature

```
[ ] Tutti i commit seguono Conventional Commits (o sono almeno descrittivi)
[ ] FEATURE.md aggiornato con decisioni prese durante lo sviluppo
[ ] Spostare FEATURE.md -> docs/features/<nome>.md oppure eliminarlo
[ ] Lanciare run-with-ai.py --promote per aggiornare CHANGELOG e README
[ ] Aprire PR / eseguire merge gitflow standard (vedi WORKFLOW-GITFLOW.md)
```

Riferimenti

- [Conventional Commits v1.0.0](#) — specifica completa
- [Keep a Changelog](#) — formato CHANGELOG usato dal generatore
- [GitFlow Workflow](#) — modello di branching
- [WORKFLOW-GITFLOW.md](#) — procedura GitFlow per questo progetto
- [docs/docs-generator/GUIDE.md](#) — guida al generatore documentazione